

Desenvolvimento e implantação de aplicações web baseadas em IA



Tempo Estimado: 60 minutos

Visão Geral

Neste projeto, utilizamos as bibliotecas de IA Watson incorporadas para criar uma aplicação que realiza análise de sentimento em um texto fornecido. Em seguida, implantamos a referida aplicação na web usando o framework Flask.

Diretrizes do projeto

Para a conclusão deste projeto, você terá que completar as seguintes 8 tarefas, com base no conhecimento que você adquiriu ao longo do curso.

Tarefas e objetivos:

- Tarefa 1: Clonar o repositório do projeto
- Tarefa 2: Criar uma aplicação de análise de sentimento usando a biblioteca NLP do Watson
- Tarefa 3: Formatizar a saída da aplicação
- Tarefa 4: Empacotar a aplicação
- Tarefa 5: Executar testes unitários na sua aplicação
- Tarefa 6: Implantar como aplicação web usando Flask
- Tarefa 7: Incorporar tratamento de erros
- Tarefa 8: Executar análise de código estático

Vamos começar!

Sobre as bibliotecas de IA Watson embutíveis

Neste projeto, você usará bibliotecas embutíveis para criar uma aplicação Python alimentada por IA.

[Bibliotecas de IA Watson embutíveis](#) incluem a biblioteca NLP, a biblioteca de conversão de texto em fala e a biblioteca de conversão de fala em texto. Essas bibliotecas podem ser incorporadas e distribuídas como parte da sua aplicação. Para sua conveniência, essas bibliotecas já estão pré-instaladas no Skills Network Labs Cloud IDE para uso neste projeto.

A biblioteca NLP inclui funções para análise de sentimentos, detecção de emoções, classificação de texto, detecção de idiomas, etc. entre outras. A biblioteca de conversão de fala em texto contém funções que realizam o serviço de transcrição e geram texto escrito a partir de áudio falado. A biblioteca de conversão de texto em fala gera áudio com som natural a partir de texto escrito. Todas as funções disponíveis em cada uma dessas bibliotecas chamam modelos de IA pré-treinados que estão todos disponíveis nos servidores do Cloud IDE, acessíveis a todos os usuários gratuitamente.

Essas bibliotecas também podem ser acessadas através dos seus sistemas pessoais. As diretrizes para isso estão disponíveis na página da biblioteca de IA Watson.

Tarefa 1: Clonar o repositório do projeto

O repositório do Github do projeto está disponível na URL mencionada abaixo.

<https://github.com/ibm-developer-skills-network/zzrjt-practice-project-emb-ai.git>

- Abra um novo Terminal e crie o diretório `practice_project` usando o comando `mkdir` e mude o diretório atual para `practice_project` usando o comando `cd`.

```
mkdir practice_project
cd practice_project
```

- Clone este repositório do GitHub, usando o terminal do Cloud IDE para o seu projeto em uma pasta chamada `practice_project`.

- ▶ Clique aqui para uma dica
- ▶ Clique aqui para a solução

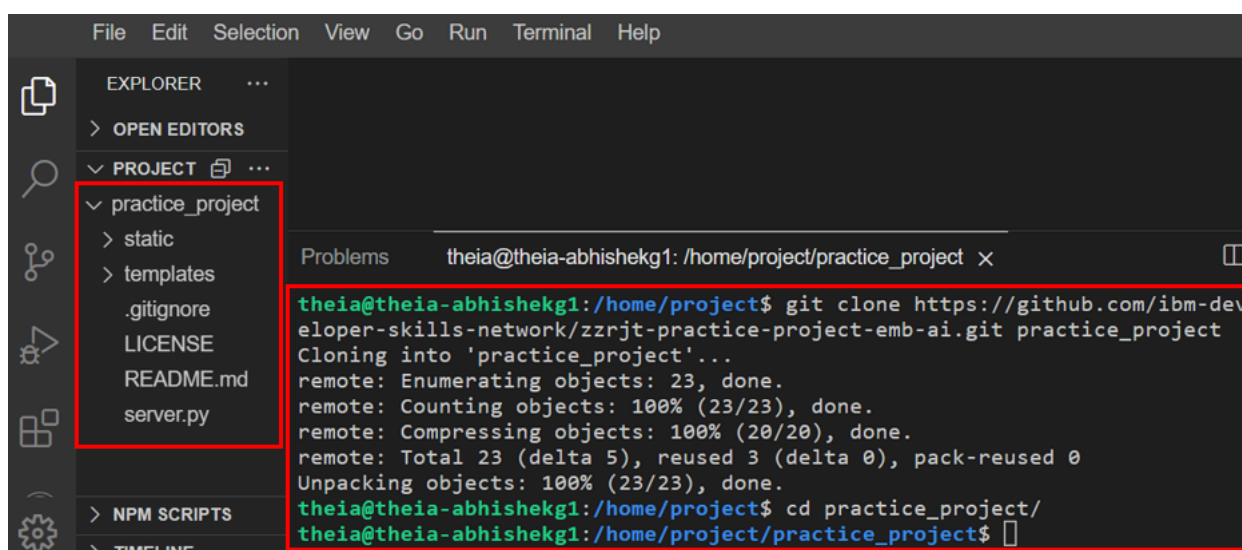
- Certifique-se de que o Python 3.11 e as bibliotecas necessárias estão disponíveis:

```
python3.11 -V
pip3.11 show requests flask pylint
```

- Instale as bibliotecas que estão faltando:

```
python3.11 -m pip install requests flask pylint
```

- Ao concluir, a aba do projeto deve ter a estrutura de pastas conforme mostrado na imagem.



Tarefa 2: Criar um aplicativo de análise de sentimentos usando a biblioteca Watson NLP

A análise de sentimentos em NLP é a prática de usar computadores para reconhecer sentimentos ou emoções expressas em um texto. Através do NLP, a análise de sentimentos categoriza palavras como positivas, negativas ou neutras.

A análise de sentimentos é frequentemente realizada em dados textuais para ajudar as empresas a monitorar a percepção da marca e dos produtos no feedback dos clientes, além de entender as necessidades dos consumidores. Isso ajuda a captar a atitude e o humor do público em geral, o que pode reunir informações valiosas sobre o contexto.

Para criar o aplicativo de análise de sentimentos, faremos uso das Bibliotecas de IA Embutidas da Watson. Como as funções dessas bibliotecas já estão implantadas no servidor Cloud IDE, não há necessidade de importar essas bibliotecas para nosso código. Em vez disso, precisamos enviar uma solicitação POST para o modelo relevante com o texto necessário e o modelo enviará a resposta apropriada.

Um código de exemplo para tal aplicativo poderia ser

```
import requests
def <function_name>(<input_args>):
    url = '<relevant_url>'
    headers = {<header_dictionary>}
    myobj = {<input_dictionary_to_the_function>}
    response = requests.post(url, json = myobj, headers=header)
    return response.text
```

Nota: A resposta das funções NLP do Watson vem na forma de um objeto. Para acessar os detalhes da resposta, podemos usar o atributo `text` do objeto chamando `response.text` e fazer a função retornar a resposta como texto simples.

Para este projeto, você usará a função de Análise de Sentimento baseada em BERT da Biblioteca NLP do Watson. Para acessar essa função, a URL, os cabeçalhos e o formato json de entrada são os seguintes.

```
URL: 'https://sn-watson-sentiment-bert.labs.skills.network/v1/watson.runtime.nlp.v1/NlpService/SentimentPredict'  
Headers: {"grpc-metadata-mm-model-id": "sentiment_aggregated-bert-workflow_lang_multi_stock"}  
Input json: { "raw_document": { "text": text_to_analyse } }
```

Aqui, `text_to_analyse` está sendo usado como uma variável que contém o texto escrito que deve ser analisado.

Nesta tarefa, você precisa criar um novo arquivo chamado `sentiment_analysis.py` na pasta `practice_project`. Neste arquivo, escreva a função para executar a análise de sentimentos usando a função de Análise de Sentimentos Watson NLP BERT, conforme discutido acima. Vamos chamar essa função de `sentiment_analyzer`. Suponha que o texto a ser analisado seja passado para a função como um argumento e seja armazenado na variável `text_to_analyse`.

► [Clique aqui para a solução](#)

Este aplicativo pode agora ser chamado usando o shell do Python. Para testar o aplicativo, abra um shell do Python usando `python3.11` para abrir o shell do python no diretório atual, ou seja, `practice_project`. *Certifique-se de que o diretório atual é `practice_project`.*

```
python3.11
```

No shell do python, importe a função `sentiment_analyzer`.

► [Clique aqui para dica](#)
► [Clique aqui para solução](#)

Após a importação bem-sucedida, teste sua aplicação com o texto “Eu amo essa nova tecnologia.”

```
sentiment_analyzer("I love this new technology")
```

O resultado esperado é conforme mostrado na imagem abaixo. Para sair do shell do python, pressione `Ctrl+Z` ou digite `exit()`.

```
>>> sentiment_analyzer("I love this new technology")  
{  
  "documentSentiment": {  
    "score": 0.996954,  
    "label": "SENT_POSITIVE",  
    "mixed": false,  
    "sentiment": "positive",  
    "text": "I love this new technology",  
    "sentimentprob": {  
      "positive": 0.9941229,  
      "neutral": 0.0058771,  
      "negative": 0.0  
    }  
  },  
  "targetedSentiments": {  
    "targetedSentiments": {},  
    "producerId": {  
      "name": "Aggregated Sentiment  
Id": {  
      "name": "Aggregated Sentiment Workflow",  
      "version": "0.0.1"}  
    }  
  }  
}
```

Isso conclui a Tarefa 2. Note que na saída, as informações relevantes para nós são apenas o `label` e o `score`. Na próxima tarefa, você extrairá essas informações dessa saída.

Tarefa 3: Formatar a saída da aplicação

A saída da aplicação criada está na forma de um dicionário, mas foi formatada como texto. Para acessar partes relevantes dessa saída, precisamos primeiro converter esse texto em um dicionário. Como dicionários são o sistema de formatação padrão para arquivos JSON, utilizamos a biblioteca embutida do Python `json`.

Vamos ver como isso funciona.

Primeiro, em um shell Python, importe a biblioteca `json`.

```
import json
```

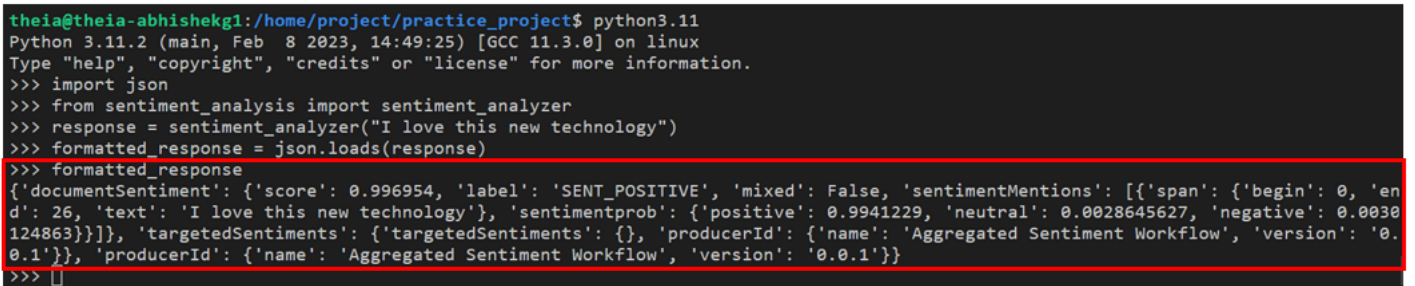
Em seguida, execute a função `sentiment_analyzer` para o texto “I love this new technology”, assim como na Tarefa 2, e armazene a saída em uma variável chamada `response`.

```
from sentiment_analysis import sentiment_analyzer
response = sentiment_analyzer("I love this new technology")
```

Agora, passe a variável `response` como um argumento para a função `json.loads` e salve a saída em `formatted_response`. Imprima `formatted_response` para ver a diferença na formatação.

```
formatted_response = json.loads(response)
print(formatted_response)
```

A saída esperada dos passos mencionados acima é mostrada na imagem abaixo.

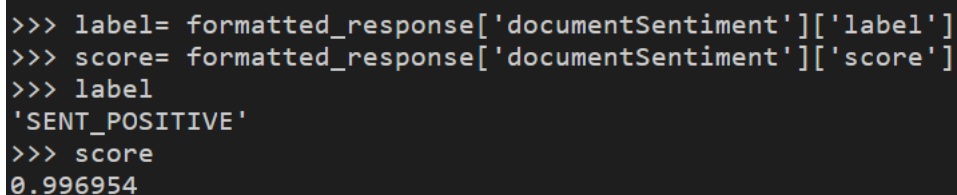


```
theia@theia-abhishekg1:/home/project/practice_project$ python3.11
Python 3.11.2 (main, Feb 8 2023, 14:49:25) [GCC 11.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import json
>>> from sentiment_analysis import sentiment_analyzer
>>> response = sentiment_analyzer("I love this new technology")
>>> formatted_response = json.loads(response)
>>> formatted_response
{'documentSentiment': {'score': 0.996954, 'label': 'SENT_POSITIVE', 'mixed': False, 'sentimentMentions': [{'span': {'begin': 0, 'end': 26, 'text': 'I love this new technology'}, 'sentimentprob': {'positive': 0.9941229, 'neutral': 0.0028645627, 'negative': 0.0030124863}}]}, 'targetedSentiments': {'targetedSentiments': {}, 'producerId': {'name': 'Aggregated Sentiment Workflow', 'version': '0.0.1'}}, 'producerId': {'name': 'Aggregated Sentiment Workflow', 'version': '0.0.1'}}
>>> []
```

Note que a ausência de aspas simples de cada lado da resposta indica que isso não é mais um texto, mas sim um dicionário. Para acessar as informações corretas deste dicionário, precisamos acessar as chaves de forma apropriada. Como esta é uma estrutura de dicionário aninhado, ou seja, um dicionário de dicionários, as seguintes instruções precisam ser usadas para obter os rótulos e as saídas de pontuação desta resposta.

```
label = formatted_response['documentSentiment']['label']
score = formatted_response['documentSentiment']['score']
```

Verifique o conteúdo de `label` e `score` para verificar a saída.



```
>>> label= formatted_response['documentSentiment']['label']
>>> score= formatted_response['documentSentiment']['score']
>>> label
'SENT_POSITIVE'
>>> score
0.996954
```

Agora, para a Tarefa 3, incorpore a técnica mencionada acima e faça alterações no arquivo `sentiment_analysis.py`. A saída esperada ao chamar a função `sentiment_analyzer` deve agora ser um dicionário com 2 chaves, `label` e `score`, cada uma tendo o valor apropriado extraído da resposta da função Watson NLP. Verifique suas alterações testando a função modificada em um shell python.

► Clique aqui para a solução

Ao concluir, a saída esperada da função é mostrada na imagem abaixo.

```
theia@theia-abhishekg1:/home/project/practice_project$ python3.11
Python 3.11.2 (main, Feb 8 2023, 14:49:25) [GCC 11.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from sentiment_analysis import sentiment_analyzer
>>> sentiment_analyzer("I love this new technology")
{'label': 'SENT_POSITIVE', 'score': 0.996954}
```

Para sair do shell python, digite `exit()` ou pressione `Ctrl+Z`.

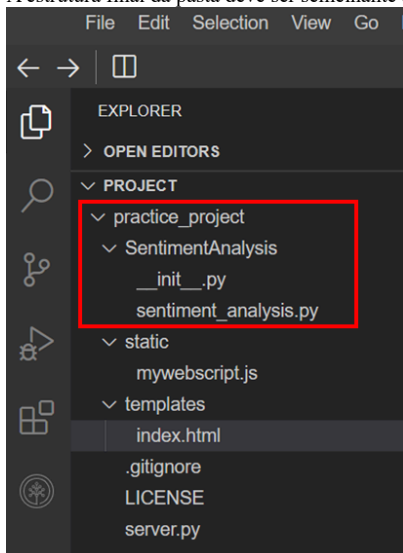
Tarefa 4: Empacotar a aplicação

Nesta tarefa, você deve empacotar a aplicação final que criou nas tarefas 2 e 3.

Vamos manter o nome do pacote como `SentimentAnalysis`. Os passos envolvidos no empacotamento são:

1. Crie uma pasta no diretório de trabalho, com o nome do pacote.
 - ▶ [Clique aqui para dica](#)
 - ▶ [Clique aqui para solução](#)
2. Mova o código da aplicação (ou seja, o módulo) para a pasta do pacote.
 - ▶ [Clique aqui para dica](#)
 - ▶ [Clique aqui para solução](#)
3. Crie um novo arquivo chamado `__init__.py` dentro da pasta do pacote para referenciar o módulo.
 - ▶ [Clique aqui para dica](#)
 - ▶ [Clique aqui para solução](#)

A estrutura final da pasta deve ser semelhante à mostrada na imagem abaixo.



`SentimentAnalysis` é agora um pacote válido e pode ser importado em qualquer arquivo deste projeto.

Para testar isso, execute um shell Python no terminal e tente importar a função `sentiment_analyzer` do pacote.

- ▶ [Clique aqui para a dica](#)
- ▶ [Clique aqui para a solução](#)

Nenhuma mensagem de erro recebida após a instrução de importação indicaria que o pacote está agora pronto para uso. Teste a função executando a seguinte instrução no shell.

```
sentiment_analyzer("This is fun.")
```

A saída recebida seria parecida com a mostrada abaixo.

```
>>> sentiment_analyzer("This is fun.")
{'label': 'SENT_POSITIVE', 'score': 0.997183}
```

Para sair do shell do python, digite `exit()` ou pressione `Ctrl+Z`.

Tarefa 5: Executar testes unitários em sua aplicação

Agora que temos uma aplicação funcional, é necessário que executemos testes unitários em alguns casos de teste para verificar a validade de suas saídas.

Para executar testes unitários, precisamos criar um novo arquivo que chame a função da aplicação necessária do pacote e teste-a para um par de texto e saída conhecidos.

Para isso, complete os seguintes passos.

1. Crie um novo arquivo na pasta `practice_project`, chamado `test_sentiment_analysis.py`.
2. Neste arquivo, importe a função `sentiment_analyzer` do pacote `SentimentAnalysis`. Também importe a biblioteca `unittest`.

► [Clique aqui para a solução](#)

3. Crie a classe de teste unitário. Vamos chamá-la de `TestSentimentAnalyzer`. Defina `test_sentiment_analyzer` como a função para executar os testes unitários.

► [Clique aqui para a solução](#)

4. Defina 3 testes unitários na função mencionada e verifique a validade dos seguintes pares de declaração - rótulos.
“I love working with Python”: “SENT_POSITIVE”
“I hate working with Python”: “SENT_NEGATIVE”
“I am neutral on Python”: “SENT_NEUTRAL”

► [Clique aqui para dica](#)

► [Clique aqui para a solução](#)

5. Chame os testes unitários.

► [Clique aqui para a solução](#)

Agora que o arquivo está pronto, execute o arquivo para realizar os testes unitários. Após a execução bem-sucedida, a saída deste arquivo deve ser conforme mostrado na imagem abaixo.

```
theia@theia-abhishek1:/home/project/practice_project$ python3.11 test_sentiment_analysis.py
.
-----
Ran 1 test in 0.501s

OK
```

Tarefa 6: Implantar como aplicação web usando Flask

Agora que a aplicação está pronta, é hora de implantá-la para uso através de uma interface web. Para facilitar o processo de implantação, você recebeu 3 arquivos que serão usados para esta tarefa.

- Verifique a estrutura do diretório:

```
practice_project/
├── SentimentAnalysis/
│   ├── __init__.py
│   └── sentiment_analysis.py
├── templates/
│   └── index.html
├── static/
│   └── mywebscript.js
└── server.py
```

- Este `index.html` na pasta `templates` contém o código para a interface web que foi projetada para este laboratório. Este arquivo está sendo fornecido para você na íntegra e deve ser usado como está. Você não precisa fazer nenhuma alteração neste arquivo.

A interface é mostrada na imagem.

NLP - Sentiment Analysis

Please enter the text to be analyzed

Run Sentiment Analysis

Result of Sentiment Analysis

- Ao clicar no botão `Run Sentiment Analysis` na interface html, chama este arquivo javascript `mywebscript.js` na pasta `static`, que executa uma requisição GET e pega o texto fornecido pelo usuário como entrada. Este texto, salvo em uma variável chamada `textToAnalyze`, é então enviado para o arquivo do servidor para ser enviado à aplicação. Este arquivo também está sendo fornecido a você na íntegra e deve ser usado como está. Você não precisa fazer nenhuma alteração neste arquivo.
- Abra `server.py` na pasta `practice_project`. Esta tarefa gira em torno da conclusão deste arquivo. Você pode completar este arquivo seguindo os 5 passos a seguir.

a. Importe as bibliotecas e funções relevantes

Neste arquivo, você precisará da biblioteca Flask junto com sua função `render_template` (para implantar o arquivo HTML) e da função `request` (para iniciar a requisição GET da página web).

Você também precisará importar a função `sentiment_analyzer` do pacote `SentimentAnalysis`.

Adicione as linhas de código relevantes, importando as funções mencionadas, em `server.py`.

► [Clique aqui para a solução](#)

b. Inicie o app Flask com o nome `Sentiment Analyzer`

Use o conhecimento adquirido no Módulo 2 deste curso e adicione a declaração ao `server.py`, que inicia a aplicação e a nomeia como `Sentiment Analyzer`.

► [Clique aqui para a solução](#)

c. Defina a função `sent_analyzer`

O propósito desta função é duplo. Primeiro, a função deve enviar uma requisição GET para a interface HTML para receber o texto de entrada. Note que a requisição GET deve referenciar a variável `textToAnalyze` conforme definida no arquivo `mywebscript.js`. Armazene o texto recebido em uma variável `text_to_analyze`. Agora, como segunda função, chame sua aplicação `sentiment_analyzer` com `text_to_analyze` como argumento.

Além disso, formate a saída retornada da função em um texto formal. Por exemplo:
O texto fornecido foi identificado como POSITIVO com uma pontuação de 0.99765.

► [Clique aqui para uma dica](#)
► [Clique aqui para a solução](#)

Nota: A função usa o decorador Flask `@app.route("/sentimentAnalyzer")` conforme referenciado no arquivo `mywebscript.js`.

d. Renderize o template HTML usando `render_index_page`

Esta função deve simplesmente executar a função `render_template` no template HTML, `index.html`.

► [Clique aqui para a solução](#)

e. Execute a aplicação em `localhost:5000`

Por fim, ao executar o arquivo, execute a aplicação no host: 0.0.0.0 (ou localhost) na porta 5000.

► Clique aqui para a solução

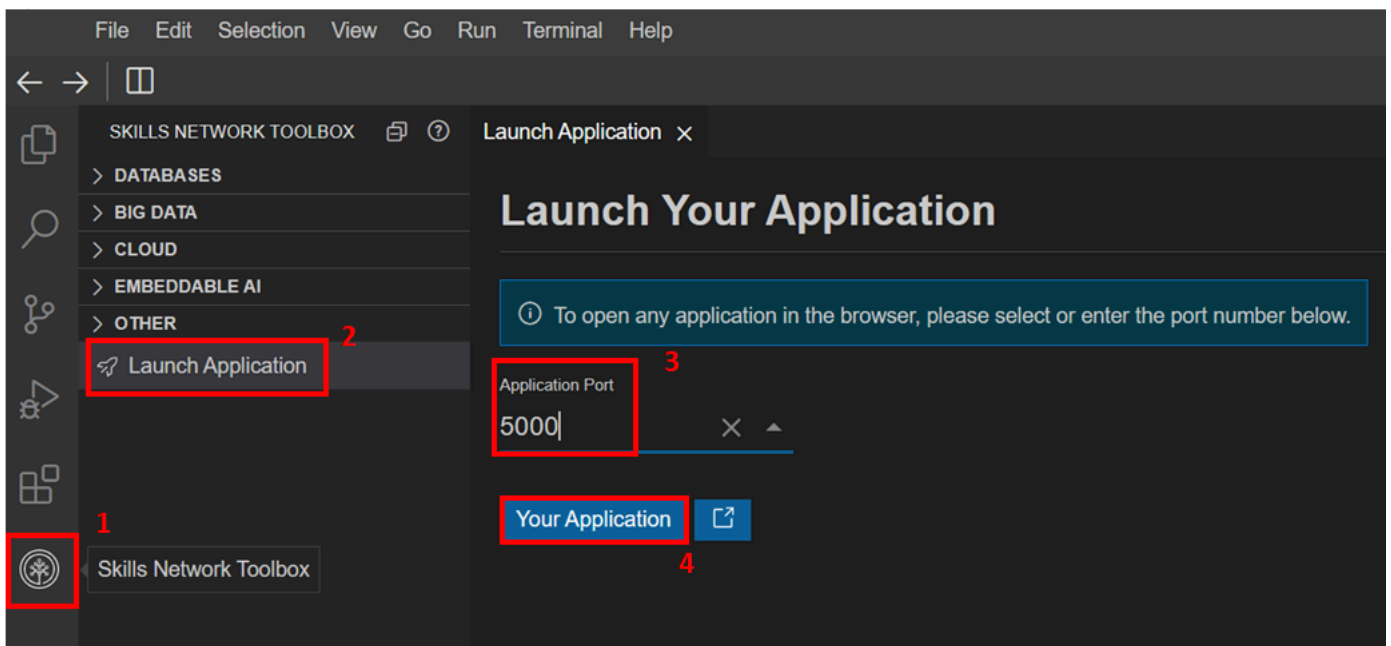
Para implantar a aplicação, execute o arquivo `server.py` a partir do terminal.

```
python3.11 server.py
```

A saída será parecida com isto.

```
theia@theia-abhishekg1:/home/project/practice_project$ python3.11 server.py
* Serving Flask app 'Sentiment Analyzer'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a pr
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.22.138.184:5000
Press CTRL+C to quit
```

O aplicativo está agora rodando em localhost:5000. Para acessar a aplicação, vá até a aba Skills Network Toolbox e clique em Launch Application. Insira a porta da aplicação como 5000 e clique em Your Application.



A interface da aplicação será aberta. Use a interface para testar sua aplicação.

server.py sentiment_analysis.py __init__.py Your Application × index.html

https://abhishekg1-5000.theianext-0-labs-prod-misc-tools-us-east-0.proxy.cognitiveclass.ai/

NLP - Sentiment Analysis using BERT

Please enter the text to be analyzed

I love this new technology

Run Sentiment Analysis

Result of Sentiment Analysis

The given text has been identified as POSITIVE with a confidence score of 0.996954.

```
Problems theia@theia-abhishekg1: /home/project/practice_project ×
127.0.0.1 - - [09/Jul/2023 15:03:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Jul/2023 15:03:51] "GET /static/mywebscript.js HTTP/1.1" 304 -
127.0.0.1 - - [09/Jul/2023 15:03:55] "GET /sentimentAnalyzer?textToAnalyze=I%20love%20this%20new%20technoI"
```

Para parar a aplicação, pressione Ctr+C.

Tarefa 7: Incorporar Tratamento de Erros

Para incorporar o tratamento de erros, precisamos identificar as diferentes formas de códigos de erro que podem ser recebidos em resposta à consulta GET iniciada pela função `sent_analyzer` em `server.py`.

Isso já faz parte das funções da Biblioteca Watson NLP e pode ser observado no console do terminal onde o código está sendo executado.

Considere a imagem mostrada abaixo.

NLP - Sentiment Analysis using BERT

Please enter the text to be analyzed

I love this new technology

Run Sentiment Analysis

Result of Sentiment Analysis

The given text has been identified as POSITIVE with a confidence score of 0.996954.

```
Problems theia@theia-abhishekg1: /home/project/practice_project ×
127.0.0.1 - - [09/Jul/2023 15:03:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Jul/2023 15:03:51] "GET /static/mywebscript.js HTTP/1.1" 304 -
127.0.0.1 - - [09/Jul/2023 15:03:55] "GET /sentimentAnalyzer?textToAnalyze=I%20love%20this%20new%20technology HTTP/1.1" 200 -
```

Os códigos indicam que a solicitação GET inicial foi bem-sucedida (200), a solicitação foi então transferida com sucesso para a Biblioteca Watson (304) e, em seguida, a solicitação GET para gerar a resposta também foi realizada com sucesso.

No caso de entradas inválidas, o sistema responde com o código de erro 500, indicando que há algo errado no lado do servidor. Entrada inválida pode ser qualquer coisa que o modelo não consiga interpretar. No entanto, nesta situação de erro, a saída desta aplicação não é atualizada.

< > https://abhishekg1-5000.theia-next-1-labs-prod-misc-tools-us-east-0.proxy.cognitiveclass.ai/

NLP - Sentiment Analysis using BERT

Please enter the text to be analyzed

asda!ka skd;alk a;lksda

Run Sentiment Analysis

Result of Sentiment Analysis

The given text has been identified as POSITIVE with a confidence score of 0.996954.

```
Problems theia@theia-abhishekg1: /home/project/practice_project x
File "/home/project/practice_project/SentimentAnalysis/sentiment_analysis.py", line 10, in sentiment_analyzer
    label = formatted_response['documentSentiment']['label']
KeyError: 'documentSentiment'
127.0.0.1 - - [10/Jul/2023 15:46:39] "GET /sentimentAnalyzer?textToAnalyze=asda!ka%20skd;alk%20a;lksda HTTP/1.1" 500 -
```

Note que a saída na interface é a mesma de antes, o texto sendo analisado é um texto aleatório e as bibliotecas de IA da Watson estão gerando um erro 500, confirmando que o modelo não conseguiu processar a solicitação.

Para corrigir esse bug em nossa aplicação, precisamos estudar a resposta recebida da função da biblioteca de IA da Watson, quando o servidor gera o erro 500. Para testar isso, precisamos refazer os passos tomados na Tarefa 2 e testar a biblioteca de IA da Watson com uma entrada de string inválida.

Abra um shell Python no terminal e execute os seguintes comandos para verificar a saída necessária após atualizar o arquivo sentiment_analysis.py com o abaixo.

```
import requests
# Define the URL for the sentiment analysis API
url = "https://sn-watson-sentiment-bert.labs.skills.network/v1/watson.runtime.nlp.v1/NlpService/SentimentPredict"
# Set the headers with the required model ID for the API
headers = {"grpc-metadata-mm-model-id": "sentiment_aggregated-bert-workflow_lang_multi_stock"}
# Define the first payload with nonsensical text to test the API
myobj = { "raw_document": { "text": "as987da-6s2d aweadsa" } }
# Make a POST request to the API with the first payload and headers
response = requests.post(url, json=myobj, headers=headers)
# Print the status code of the first response
print(response.status_code)
# Define the second payload with a meaningful text to test the API
myobj = { "raw_document": { "text": "Testing this application for error handling" } }
# Make a POST request to the API with the second payload and headers
response = requests.post(url, json=myobj, headers=headers)
# Print the status code of the second response
print(response.status_code)
```

A resposta do console aparece como mostrado na imagem abaixo. As caixas vermelhas indicam o texto inválido e seu código de status recebido, e as caixas amarelas indicam o texto válido e seu código de status recebido.

```
theia@theia-abhishekg1:/home/project/practice_project$ python3.11
Python 3.11.2 (main, Feb 8 2023, 14:49:25) [GCC 11.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import requests
>>> url = "https://sn-watson-sentiment-bert.labs.skills.network/v1/watson.runtime.nlp.v1/NlpService/SentimentPredict"
>>> headers = {"grpc-metadata-mm-model-id": "sentiment_aggregated-bert-workflow_lang_multi_stock"}
>>> myobj = { "raw_document": { "text": "as987da-6s2d aweadsa" } }
>>> response = requests.post(url, json = myobj, headers=headers)
>>> print(response.status_code)
500
>>>
>>> myobj = { "raw_document": { "text": "Testing this application for error handling" } }
>>> response = requests.post(url, json = myobj, headers=headers)
>>> print(response.status_code)
200
>>>
```

Isso permite que você modifique o aplicativo de tal forma que possamos enviar diferentes saídas para diferentes códigos de status.

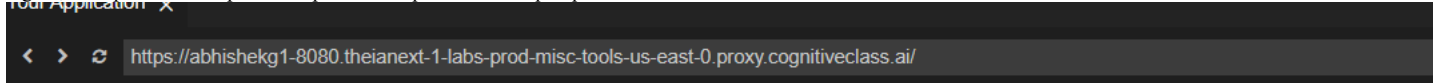
Na primeira parte desta tarefa, você deve modificar a função `sentiment_analyzer()` para retornar tanto `label` quanto `score` como `None` em caso de entrada de texto inválida.

- ▶ Clique aqui para uma dica
- ▶ Clique aqui para a solução

Agora, em `server.py`, a resposta a ser enviada ao console também deve ser diferente para os tipos de entrada válidos e inválidos. Para entrada inválida, deixe o console imprimir `Entrada inválida! Tente novamente.`

- ▶ Clique aqui para uma dica
- ▶ Clique aqui para a solução

Agora, seu aplicativo é capaz de responder adequadamente a qualquer forma de entrada.



NLP - Sentiment Analysis using

Please enter the text to be analyzed

Kayqa huk prueba kay ruwanapa ruwayninta qhawanapaq.

Run Sentiment Analysis

Result of Sentiment Analysis

Invalid input ! Try again.

Tarefa 8: Executar análise de código estático

Finalmente, na Tarefa 8, verificamos a qualidade das suas habilidades de codificação de acordo com as diretrizes PEP8, executando uma análise de código estático.

Normalmente, isso é feito no momento da embalagem e dos testes unitários da aplicação. No entanto, mantivemos essa etapa no final deste projeto, uma vez que os códigos foram atualizados em todas as tarefas anteriores. Agora que seus arquivos para este projeto estão prontos, vamos testá-los quanto à conformidade com as diretrizes PEP8.

O primeiro passo nesse processo é instalar a biblioteca `PyLint` usando o terminal.

- ▶ Clique aqui para a solução

Em seguida, use `pylint` para executar a análise de código estático em `server.py`.

- ▶ Clique aqui para a solução

Se todos os aspectos do guia PEP8 foram incorporados ao seu código, a pontuação gerada deve ser 10/10. Caso contrário, siga as instruções fornecidas pela biblioteca `pylint` para modificar o código corretamente.

```
theia@theia-abhishekg1:/home/project/practice_project$ pylint server.py
-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Isso conclui o projeto prático.

(Opcional) Exercícios Adicionais

Os aprendizes interessados podem tentar os seguintes exercícios por conta própria, para uma melhor compreensão dos conceitos aprendidos neste projeto. Nenhuma solução está sendo fornecida para esses exercícios. No entanto, sintá-se à vontade para discutir e compartilhar suas soluções com seus colegas nos fóruns de discussão do curso.

1. Execute uma análise de código estático em `sentiment_analysis.py`. Tente alcançar uma pontuação de 10/10. Dica: *Docstrings*

2. Teste a capacidade da sua aplicação em lidar com frases de idiomas diferentes do inglês, por exemplo, francês, alemão, etc. Veja se a aplicação responde com um erro de texto inválido.
3. Atualmente, se a aplicação for executada SEM fornecer uma entrada, ou seja, deixando o texto em branco, o modelo ainda lança o mesmo erro de texto inválido. Tente incluir um caso especial, onde uma entrada em branco receba uma mensagem de erro diferente.

Conclusão

Parabéns por completar este projeto.

Com a conclusão deste projeto, você:

1. Criou uma aplicação de análise de sentimentos baseada em IA usando as bibliotecas incorporadas do Watson NLP.
2. Formatou a saída recebida da função da biblioteca Watson NLP para extrair informações relevantes.
3. Empacotou a aplicação e a tornou importável para qualquer código Python para uso.
4. Executou testes unitários na aplicação e verificou a validade de suas saídas para diferentes entradas.
5. Implantou a aplicação usando o framework Flask.
6. Incorporou a capacidade de tratamento de erros na aplicação, de modo que um código de resposta 500 receba uma resposta apropriada da aplicação.
7. Executou análise de código estático nos arquivos de código para confirmar sua conformidade com as diretrizes PEP8.

© IBM Corporation 2023. Todos os direitos reservados.